

Performance Analysis of Sparse Data Storage Implementations

Alessia Ciccaglione

#####

Politecnico di Torino

Computational Linear Algebra for Large Scale Problems, A.A. 2024-2025

Abstract—Sparse data storage is important in many applications where matrices often contain a significant proportion of zero values.

This paper focus on the analysis of the performance of different sparse matrix storage formats (COO, CSR, CSC, DOK, LIL and DIA) using the MovieLens dataset. [3]

The MovieLens dataset is commonly used in recommendation systems and provides ratings of 1682 movies from 943 users. We evaluate and compare matrix creation times, matrix-vector multiplication times Singular Value Decomposition (SVD) execution times and memory usage for each format.

The results reveal significant performance variations across storage formats, highlighting the trade-offs between computational efficiency and memory usage.

I. INTRODUCTION

In many application, such as user-item interactions, text data, and network graph, the efficient storage and manipulation of sparse matrices is important. Sparse matrices often contain a high proportion of zero elements, making traditional storage methods inefficient in terms of both memory usage and computational performance.

To address this issue, efficient sparse storage formats have been developed to record only the non-zero elements. These formats enable faster computations, reduce memory overhead, and facilitate operations like matrix-vector multiplication and Singular Value Decomposition (SVD).

Sparse matrices are also central to recommendation systems, where users typically rate a small fraction of items, resulting in sparse matrices which represent user-item interactions.

The execution times can be influenced by many factors, such as the hardware specifications (e.g., CPU, RAM), the environment (e.g., operating system, background processes), and the specific configuration of the software used. For this reason, when running the same code on different machines or under different conditions, the results may vary, and performance may differ accordingly. However, the overall trend and the most significant differences tend to remain the same.

II. THE MOVIELENS DATASET

To compare the performances of different sparse matrices format we used the MovieLens dataset with 100k ratings of

943 users on 1682 movies. Each user has rated at least 20 movies.

The rows of the dataset correspond to users, the columns correspond to movies, and the entries denote the ratings.

From the scatter plot in Figure 1, we observe the high sparsity of the dataset, with over 93% of the entries being zero. This enlightens the importance of efficient sparse matrix storage techniques in this situation.

The plot does not reveal any specific pattern in the distribution of non-zero entries. Most users have rated only a small number of movies. The darker regions in the plot represent either movies that have been rated by many users or users who have rated many movies.

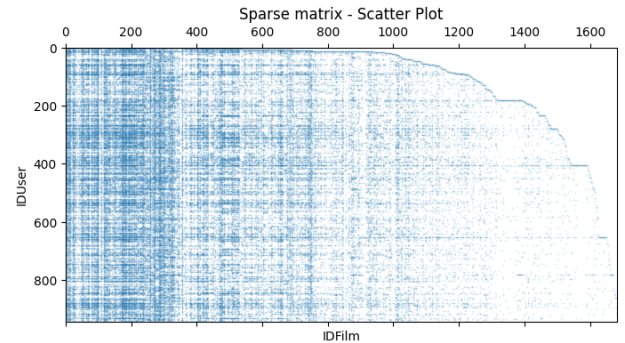


Fig. 1. Scatter plot visualizing the sparsity of the MovieLens dataset.

III. SPARSITY AND STORAGE SCHEMES

We consider the following storage formats for the analysis:

- Coordinate List (COO): This format stores the row indexes, column indexes, and non-zero values in three separate arrays. It is simple to implement and efficient for constructing sparse matrices but not optimal for arithmetic operations.
- Compressed Sparse Row (CSR): CSR stores the non-zero values, column indexes, and row pointers in three arrays. It is efficient for row slicing and other operations focusing on accessing rows like multiplying the matrix by a column vector ($A * x$).

- **Compressed Sparse Column (CSC):** Similar to CSR, CSC stores the non-zero values, row indexes, and column pointers. It is more efficient for operations that focus on columns, such as matrix-vector multiplication where the matrix is multiplied by a column vector from the left ($x * A$) or a row vector from the right ($A * x.T$).
- **Dictionary of Keys (DOK):** DOK stores the non-zero values in a dictionary, where the keys are the coordinates of the non-zero elements. This format is highly flexible and easy to update but is less efficient in terms of memory usage for large matrices.
- **List of Lists (LIL):** LIL represents a sparse matrix as a list of rows, where each row is a list of pairs containing the column index and the corresponding value. It is easy to modify and update but is less efficient for large matrices and matrix-vector multiplication.
- **Diagonal Storage (DIA):** DIA is designed for sparse matrices where non-zero elements are concentrated along a small number of diagonals. It stores the matrix using two arrays. A one-dimensional array containing the diagonal indexes of the non-zero diagonal and a two-dimensional array where each column corresponds to a diagonal specified by IOFF. The elements of each diagonal are stored in lexicographic order, and unused entries in DIA are left empty. It is highly efficient for diagonally dominant matrices and operations that only involve those diagonals.

The analysis excluded the Ellpack-Itpack, MSR (Modified Sparse Row) and MSC (Modified Sparse Columns) formats due to compatibility issues.

Specifically, the fixed-width structure of the Ellpack-Itpack method does not support implementing SVD, making comparisons between formats challenging.

MSR and MSC formats are efficient only for square matrices with only non-zero values on the main diagonal. The matrix related with the MovieLens dataset does not exhibit these characteristics.

The DIA format was only partially explored because initial tests showed that the MovieLens dataset lacked the diagonal dominance necessary to justify using this format. To verify this, we evaluated the diagonal density of the 100 principal diagonals and the analysis revealed a diagonal density of 9.25%.

IV. ANALYSIS OF STORAGE METHODS ON THE MOVIELENS DATASET

As mentioned earlier, storing only the non-zero entries can significantly reduce memory usage and increase computational efficiency.

In this section, we focus on the performance evaluation of each storage format applied to the MovieLens dataset.

We used the MovieLens dataset to deal with a realistic workload for testing.

A. Implementation of Storage Methods

We used the 'scipy.sparse' library in Python to implement the following formats: Coordinate format, Compressed Sparse Row, Compressed Sparse Column, Dictionary of Keys and List of Lists.

The resulting representations include the following data structures:

- **Coordinate List (COO):**
AA-data values: [5. 3. 4. ... 3. 3. 3.]
JR-row indexes: [0 0 0 ... 942 942 942]
JC-col indexes: [0 1 2 ... 1187 1227 1329]
- **Compressed Sparse Row (CSR):**
AA-data: [5. 3. 4. ... 3. 3. 3.]
IA-indptr: [0 272 334 388 412 587 798 1201 1260 ...]
JA-indexes: [0 1 2 ... 1187 1227 1329]
- **Compressed Sparse Column (CSC):**
AA-data: [5. 4. 4. ... 2. 3. 3.]
IA-indptr: [0 452 583 673 882 968 994 1386 1605...]
JA-indexes: [0 1 4 ... 862 895 915]
- **Dictionary of Keys (DOK):**
(0, 23) 3.0
(0, 24) 4.0
:
(942, 738) 4.0
- **List of Lists (LIL):**
L0: [(0, 5.0), (1, 3.0), (2, 4.0), ..., (271, 3.0)]
L1: [(0, 4.0), (9, 2.0), (12, 4.0), ... , (315, 5.0)]
L2: [(180, 4.0), (244, 1.0), (257, 2.0), ... , (315, 5.0)]

B. Memory Usage

The following graph *Figure 2* highlights the differences in memory usage across different formats. CSR and CSC formats demonstrate better memory efficiency, while DOK and LIL require more memory due to their flexible structures.

The bar chart illustrates the memory consumption (in KB) required by different sparse matrix storage formats: COO, CSR, CSC, DOK, and LIL. The findings reveal that:

- **Coordinate format** and **Dictionary of Keys** demand the highest memory, around 1600 KB, making them the least memory-efficient formats.
- **Compressed Sparse Row (CSR)** and **Compressed Sparse Column (CSC)** are more memory-efficient, with both consuming slightly over 1175 KB.
- **List of Lists (LIL)** consumes the least memory (1171.86 KB). Despite its low memory usage, the overhead of maintaining separate lists for each row could limit its usability in certain applications.

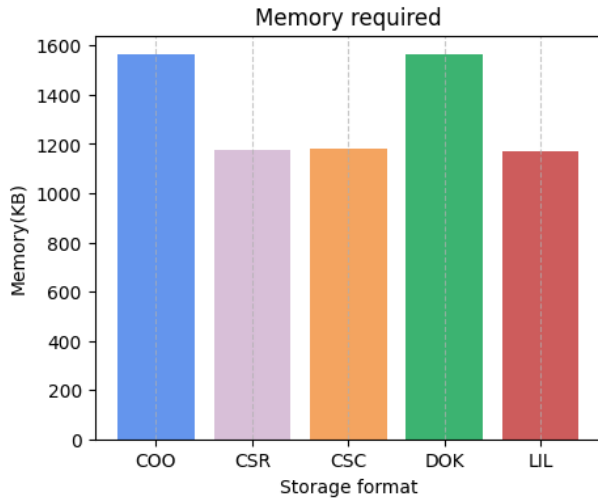


Fig. 2. Memory usage for different sparse formats.

- **Coordinate format (COO)** and **Dictionary of Keys (DOK)** require the highest memory (1562.48 KB), making them the least memory-efficient formats.
- **Compressed Sparse Row (CSR)**, **Compressed Sparse Column (CSC)**, and **List of Lists (LIL)** have similar and lower memory requirements, all around 1175 KB. These formats are more memory-efficient compared to COO and DOK, with LIL being the most efficient (1171.86 KB).

These results highlight that CSR, CSC, and LIL are all suitable choices for memory-constrained environments due to their comparable memory usage. On the other hand, COO and DOK are less optimal due to their high memory demands.

C. Performance Analysis of Matrix Creation Times

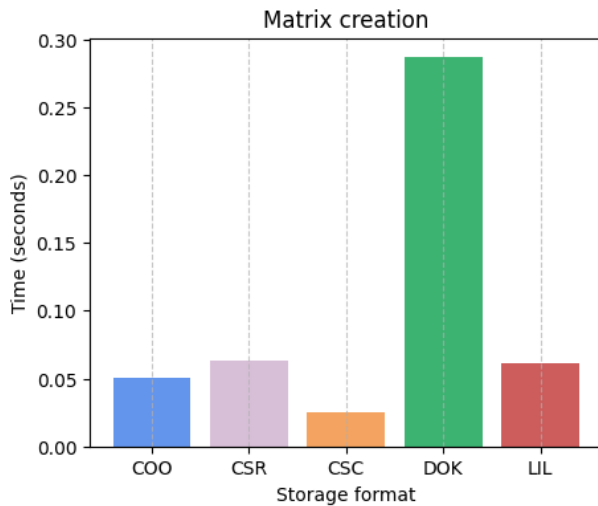


Fig. 3. Creation time for different sparse matrix formats.

The bar chart displays the matrix creation times (in seconds) for various sparse matrix storage formats: COO, CSR, CSC, DOK, and LIL. Key observations are:

- **Dictionary of Keys** has the longest matrix creation time making it the least efficient format for this operation.
- **Compressed Sparse Column** has the fastest creation time, equal to 0.026 seconds.
- **Coordinate format, Compressed Sparse Row and List of Lists** perform similarly, with times below 0.063 seconds, indicating high efficiency.

These results emphasize that DOK is unsuitable for tasks requiring frequent matrix creation due to its significant overhead, while COO, CSR, CSC, and LIL offer comparable and efficient performance.

In conclusion, we observed that LIL, CSR, and CSC are similarly memory-efficient formats, with minor differences in their consumption. Among these, CSC provides the best balance between memory usage and creation speed, making it ideal for time-sensitive tasks. In contrast, COO and DOK are less suitable for memory-constrained or time-sensitive environments due to their significantly higher memory demands and, in the case of DOK, slower creation time.

D. Matrix by vector Multiplication

Storage schemes for sparse matrices effectively improve the complexity of matrix operations, particularly in matrix-vector multiplication.

Sparse matrices, by definition, contain a large proportion of zero elements. Without efficient storage schemes, operations on such matrices would require unnecessary processing of these zero elements, leading to wasted computational resources.

In our project, to highlight the efficiency of storage formats in matrix-vector multiplication, we conducted a comparison between the average movie ratings and the average movie ratings based on user experience.

The user experience was quantified by a score ranging from 0 to 2, calculated based on the number of movies each user had rated. We compute the multiplication between the transposed matrix representing the original rating and a vector representing the score assigned to each user. This scoring system reflects the user's engagement with the platform, with a higher score indicating more ratings and, presumably, more reliable preference data.

By evaluating the performance of various sparse matrix storage formats during matrix-vector multiplication, we were able to assess how efficiently each format handled the underlying matrix data. Specifically, the goal was to understand how the choice of storage format impacts the speed and memory usage during matrix-vector multiplication.

As shown in Figure 4 the analysis showed similar performance for most formats in this specific case (excluding DOK),

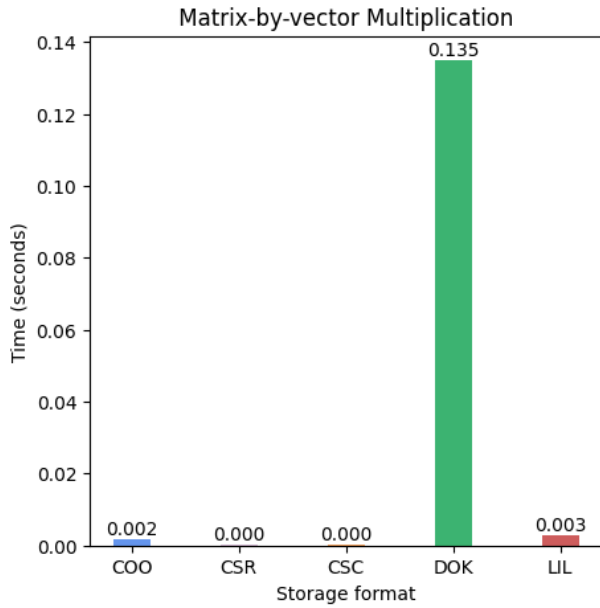


Fig. 4. Matrix by vector multiplication for different sparse matrix formats.

which can be attributed to the characteristics of the MovieLens dataset.

Performing a multiplication on the transpose of the original matrix to compute the movie scores can be optimized using the CSC (Compressed Sparse Column) format. However, as shown by the Algorithm 1, the utility of this format is tied to the presence of columns with no non-zero elements, which allows the algorithm to skip unnecessary calculations. In our case, since every movie has been rated by at least one user, all columns contain non-zero elements. This decrease the advantage of this storage method, making it comparable to other formats.

Due to the advantages that can arise from using the CSC format in this type of calculations, we further examine the application of the CSC format in matrix multiplications in detail within the Algorithm 1. The CSC format provides faster access to non-zero elements, reducing the number of operations compared to other formats, which are not optimized for column access. When multiplying a transposed matrix by a vector, it effectively involves working with the rows of the original matrix, which become columns in the transposed version. The transposed matrix retains the same CSC structure, but the data now correspond to the rows of the original matrix (i.e., columns of the transpose). During multiplication, each column (representing a row of the original matrix) can be accessed directly using compressed indexes and values in the CSC format.

V. ANALYZING OPERATION TIMES IN COLLABORATIVE FILTERING METHODS

Collaborative filtering is based on an intuitive idea we use in everyday life. When looking for a new movie to watch, we often ask friends for recommendations, especially those

Algorithm 1 Matrix-Vector Multiplication using Transposed Matrix in CSC format

Initialization: $w \leftarrow 0 \in \mathbb{R}^m$
for $i = 1, \dots, n$ **do** loop on columns
 $IA[i+1] - IA[i] = \text{non-zero entries of column } i$
if $IA[i] \neq IA[i+1]$ **then**
 $w_i \leftarrow \sum_{k=IA[i]}^{IA[i+1]-1} AA[k] \cdot v_{JA[k]}$

To compute each element of the final vector, we calculate the sum of the products between each element of the columns and the corresponding elements of the column vector.

end if
end for
return w

who share our tastes. The goal of collaborative filtering is to predict how much a user will appreciate an item they haven't yet rated, based on historical preference data collected from a community of users. Two main approaches to collaborative filtering have been developed: memory-based methods and model-based methods.

Memory-Based Collaborative Filtering

Memory-based collaborative filtering [2] assumes that users with similar historical preferences will continue to have similar preferences in the future. Ratings for items are calculated by considering the preferences of users or items in the neighborhood. Memory-based systems can be divided into two subtypes: user-based filtering and item-based filtering.

- *User-Based Filtering:* This method identifies users with similar tastes by comparing their past interactions. It then suggests items that are favored by users with similar preferences, which the target user has not yet discovered. For instance, if User A and User B have both liked several movies and User B has enjoyed an additional movie that User A hasn't watched, the system will recommend that movie to User A.
- *Item-Based Filtering:* In this method instead the system analyses the relationships between the items, identifying those that are frequently liked together. For example, if many users who liked Item X also liked Item Y, the system will recommend Item Y to users who appreciated Item X.

Model-Based Collaborative Filtering

Instead of using predefined rules, model-based collaborative [4] filtering employs models to discover hidden patterns in the data. These models predict user preferences for unseen items by training on historical interaction data between users and items.

One of the most well-known techniques in this category is SVD matrix decomposition.

A. Singular Value Decomposition - SVD

Theorem 1: Let $A \in \mathbb{C}^{m,n}$. Then, there exists a unique diagonal matrix $\Sigma \in \mathbb{R}^{m,n}$ with no zero elements on the diagonal, such that there exist $U \in \mathbb{C}^{m,m}$ and $V \in \mathbb{C}^{n,n}$ satisfying:

$$A = U\Sigma V^H \quad U^H U = I \quad V^H V = I$$

Note that if A is a real matrix, such that $A \in \mathbb{R}^{m,n}$, the previous results apply with the following modifications:

$$A = U\Sigma V^T \quad U^T U = I \quad V^T V = I$$

$$A = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} = U\Sigma V^T$$

$m \times n$

$$U = \begin{bmatrix} | & | & \dots & | \\ u_1 & u_2 & \dots & u_m \\ | & | & \dots & | \end{bmatrix} \quad V^T = \begin{bmatrix} V_1^T \\ V_2^T \\ \vdots \\ V_n^T \end{bmatrix}$$

$m \times m$ $n \times n$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_p \\ 0 & 0 & \vdots & 0 \\ 0 & 0 & \vdots & 0 \end{bmatrix}$$

$m \times n$

The diagonal entries of Σ are the **singular values** of A , typically denoted as $\sigma_1, \sigma_2, \dots, \sigma_p$, where r of these singular values are non-zero. These singular values are all positive and arranged in decreasing order. The expression $A = U\Sigma V^T$ represents the **Singular Value Decomposition** of A .

Since only r singular values are non-zero, we can focus on the matrices U_r and V_r^T , which correspond to the non-zero singular values. The remaining columns of U and V do not contribute to the representation of A and the rank of A is equal to r .

The singular value decomposition of a matrix A can be expressed as:

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T$$

The Singular Value Decomposition (SVD) offers a powerful computational technique for achieving low-rank approximations of the original data matrix, making it especially valuable in the context of recommender systems. By retaining only the

k largest singular values (where $k \ll r$) and discarding the remaining ones, we can effectively reduce the dimensionality of the data while capturing the essential latent structures present in the original dataset.

$$A = \sum_{i=1}^k \sigma_i u_i v_i^T$$

The low-rank approximation to rank k was performed using the 'svds' command from 'scipy.sparse.linalg' and $k=20$.

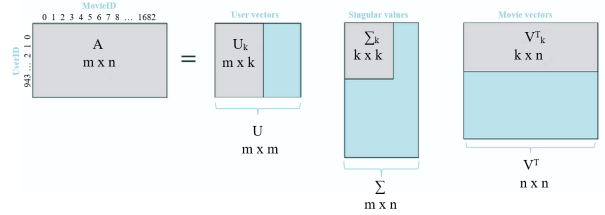


Fig. 5. Singular Value Decomposition representation on MovieLens dataset

The predicted rating [1] for user u_i on all the items is given by:

$$p_{ri} = u_i \Sigma_k V_k^T$$

where:

- p_{ri} is the vector of predicted ratings for user u_i across all items. Each element p_{ri_j} of this vector represents the predicted rating for item v_j and it's an element of the reduced matrix A_k .
- u_i is the left singular vector corresponding to user u_i , which contains the latent factors that represent the user's preferences.
- Σ_k is the diagonal matrix containing the singular values, indicating the importance of each latent factor.
- V_k^T is the matrix of right singular vectors (or item features) transposed, which contains the latent factors representing the items.

The recommendation is generated by ignoring movies that the user has already seen and sorting the predictions to obtain the top 5 movies that u_i are likely to enjoy the most.

B. Analysis of Execution Times

The utility of SVD decomposition in our project is tied to the analysis of the time required to perform the decomposition on the user-item rating sparse matrix stored in various formats. In Figure 6, the histogram comparing the execution times for these operations is presented.

The bar chart compares the time required for Singular Value Decomposition across different sparse matrix storage formats: COO, CSR, CSC, DOK, and LIL. The results show that Compressed Sparse Column is the most efficient format, completing the operation in just 0.186 seconds, while Dictionary of Keys is the least efficient, taking a significantly longer

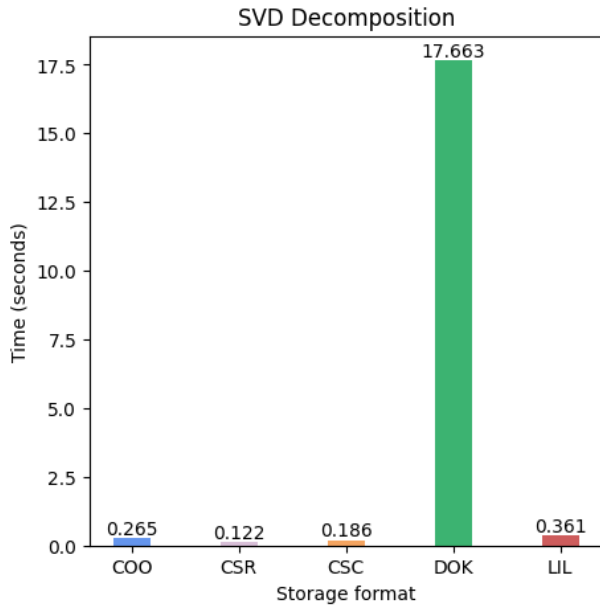


Fig. 6. Singular Value Decomposition time for different sparse matrix formats.

time of 17.663 seconds. The remaining formats, COO, CSR, and LIL, perform moderately, with times of 0.265, 0.122, and 0.361 seconds, respectively. These findings highlight the stark inefficiency of DOK for SVD, likely due to its slow indexing and lack of optimization for computational tasks. In contrast, CSC proves to be the optimal choice, offering faster access patterns and better efficiency for this type of operation.

VI. SPARSE MATRIX REORDERING: IMPACT ON COMPUTATIONAL TIMES

The structure of sparse matrices requires optimized strategies for both storage and computation. Reordering, which takes advantage of the arrangement of non-zero entries, is one of the most effective techniques for enhancing performance.

Several algorithms and strategies have been developed to modify the pattern of a sparse matrix, enhancing performance.

A. Level-Set Orderings

This class of orderings contains a number of techniques that are based on traversing the graph by level sets. A level set is defined recursively as the set of all unmarked neighbors of all the nodes of a previous level set. As soon as a level set is traversed, its nodes are marked and numbered. The neighbors of each of these nodes are then inspected. In the Cuthill-McKee ordering the nodes adjacent to a visited node are always traversed from lowest to highest degree. A very important variation of that algorithm, implemented in our project, is the reverse Cuthill-McKee. The main difference between the two algorithms is that the reverse variation reorients the matrix, creating an "arrow" pointing downward, leading to greater efficiency in preserving matrix sparsity, as can be seen in

Figure 7 The output of a Level-Set ordering, including both the

Algorithm 2 Reverse Cuthill-McKee Algorithm

Initialization: $i \leftarrow 1$, $S \leftarrow [v]$, $visited(v) \leftarrow True$, $visited(w) \leftarrow False$ for any $w \in V \setminus \{v\}$

while $i < |V| = n$ **do**
 $v \leftarrow S[i]$, $S' \leftarrow []$;
for w neighbour of v such that $visited(w) = False$ **do**
append w to S' and set $visited(w) \leftarrow True$;
end for
Sort S' with respect to degree-ascending order;
Append S' to S ;
 $i \leftarrow (i + 1)$;
end while

The Reverse Cuthill-McKee ordering is given by reorder the vertices backward:
 $S \leftarrow S[::-1]$

return S

standard and reverse Cuthill-McKee orderings, is to achieve a matrix with reduced bandwidth: minimize the number of non-zero entries in each row or column away from the diagonal to achieve better computational performance.

The bandwidth of a matrix is defined as the width of the band of non-zero entries around the diagonal. Formally, the bandwidth b of a matrix A is given by:

$$B(A) = \max_{i,j} (|i - j| : A_{ij} \neq 0)$$

where A_{ij} represents the matrix elements.

B. Independent-Set Orderings

This type of permutations operates on the independent set S of vertices from the adjacency graph G of a matrix A . An independent set $S \subseteq V$ is a subset of vertices in a graph where no two vertices in S are adjacent. Formally, for any pair of distinct vertices $u, v \in S$, there is no edge between u and v , i.e., $(u, v) \notin E$.

The Independent Set Ordering is computed in two main steps:

- 1) **Extracting an Independent Set S**
- 2) **Reordering the Matrix A :** The rows and columns of the matrix A are reordered by moving the indexes in S to the first positions, placing them in the upper-left block of the matrix.

There are various simple and computationally inexpensive heuristics for finding maximal independent sets. A common greedy approach works as follows: Traverse the nodes of the graph in a given order, if a node is not already marked, it selects the node as a new member of S . Once a node is added to S , it and all its neighboring nodes are marked. The output independent set $S = \{v_{i_1}, \dots, v_{i_s}\}$ can be used to define a permutation σ of the matrix. The permutation σ will reorder the matrix A such that the result $\sigma(A)$:

$$\sigma = \begin{pmatrix} 1 & 2 & \dots & s & s+1 & s+2 & \dots & n \\ i_1 & i_2 & \dots & i_s & j_1 & j_2 & \dots & j_{n-s} \end{pmatrix}$$

Algorithm 3 Independent Set Orderings Algorithm

Input: Adjacency graph $G = (V, E)$

Initialization: $S \leftarrow []$ empty set, $\text{marked}(v) \leftarrow \text{False}$, for each $v \in V$

Retrieving an independent set S of vertices of G .

for $i = 1, \dots, |V|$ **do**

if $\text{marked}(v_i) = \text{False}$ **then**

$S \leftarrow (S \cup \{v_i\})$

$\text{marked}(v_i) \leftarrow \text{True}$

for each w_i neighbor of v_i **do**

$\text{marked}(w_i) \leftarrow \text{True}$

end for

end if

end for

return S

Define the permutation σ and compute $A_{\text{new}} = \sigma(A)$.

$S_{\text{sorted}} \leftarrow \text{sort}(S)$

$S_c \leftarrow \{1, 2, \dots, n\} \setminus S_{\text{sorted}}$

$\sigma \leftarrow \text{concatenate}(S_{\text{sorted}}, S_c)$

for $i = 1$ to n **do**

for $j = 1$ to n **do**

$A_{\text{new}}[i, j] \leftarrow A[\sigma[i], \sigma[j]]$

end for

end for

return A_{new}, σ

such that σ starts with the indexes $i_1, i_2, \dots, i_s$, representing the vertices in the independent set S , and then continues with the indexes $j_1, j_2, \dots, j_{n-s}$, representing the vertices not included in S . The result of an Independent Set Ordering is a matrix that has a more desirable structure, particularly a diagonal upper-left block. This structure is useful for optimizing various computational methods.

C. Impact on Execution Time

The original matrix displays a seemingly random distribution of non-zero values.

Reordering techniques such as Reverse Cuthill-McKee and Independent Set Ordering modify the distribution of non-zero elements, as illustrated in *Figure 7* and *Figure 8*.

The Reverse Cuthill-McKee algorithm is designed to reduce the bandwidth of a matrix by reordering its rows and columns, which tends to create a more banded structure.

On the other hand, the independent set method focuses on selecting a set of non-adjacent vertices, which may not significantly modify the overall pattern of the matrix structure, leading to less noticeable changes compared to the original matrix.

To evaluate the impact of reordering, we constructed a sparse matrix in COO format for a square matrix of size 943×943 , as well as for the corresponding reordered versions of the original matrix. The same performance analysis as in the previous case was applied to these sparse matrices, focusing on the evaluation of creation, decomposition, and

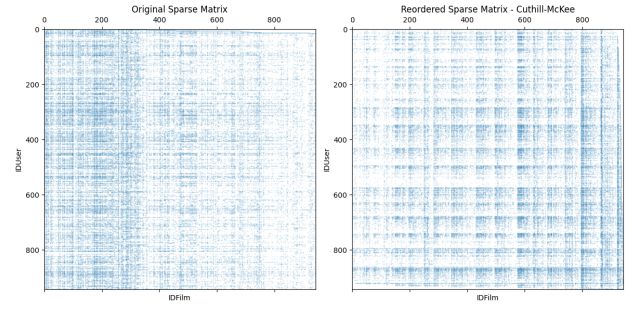


Fig. 7. Effect of the Cuthill-McKee Reordering on a Sparse Matrix.

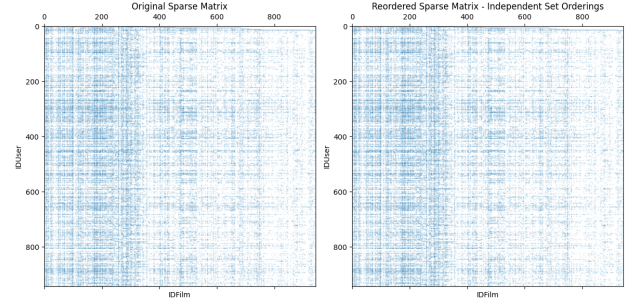


Fig. 8. Effect of the Independent Set Reordering on a Sparse Matrix.

multiplication times, which are compared in *Figure 9*.

In the context of matrix creation, we measure the time required to construct a sparse matrix in COO format starting from the original or reordered matrix. The time required for the reordering itself is not included in these measurements, as it must be added to the final evaluations. However, this reordering time depends on the specific purpose of the matrix. The Reverse Cuthill-McKee reordered matrix showed the shortest creation time, followed by the original matrix. The reordering of the matrix using the Independent Set method required the most time.

The original matrix resulted to have the fastest execution time for matrix-vector multiplication. The Reverse Cuthill-McKee reordered matrix followed closely. The Independent Set reordered matrix was the slowest.

The original matrix also resulted to be the fastest for SVD decomposition, completing in 0.1947 seconds, followed by the Independent Set reordered matrix at 0.3624 seconds, and the Reverse Cuthill-McKee reordered matrix with 0.4602 seconds.

D. Comparison on Performance Between Original and Reordered Matrices

We proceed to apply Singular Value Decomposition (SVD) to the original sparse matrix and its reordered matrices to generate movie recommendations. To compare the performances, the top 5 recommended movies for a randomly selected user were identified with each method.

Reordering the data matrix significantly influence the relationships identified by the Singular Value Decomposition algorithm.

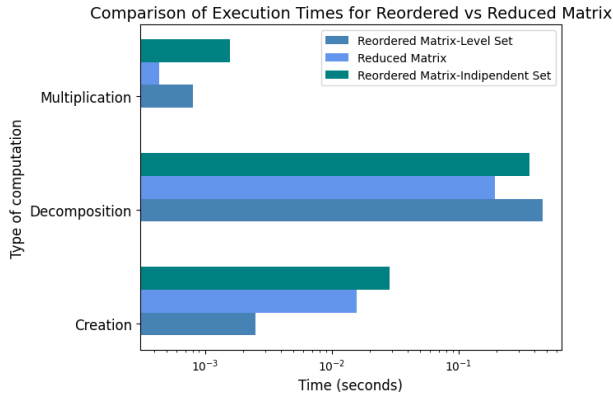


Fig. 9. Comparison of Execution Times: Original Matrix vs. Reordered Matrix both Independent and Level Set Reordering.

Both Reverse Cuthill-McKee and Independent Set Ordering in fact change how the nonzero elements are arranged within the matrix, and this alteration affects how the SVD algorithm decomposes the matrix, leading to variations in the resulting U , Σ , and V^T matrices. These differences impact the reconstructed matrix and consequently the relationships learned by the algorithm leading to generate different movie recommendations.

In this situation no significant differences were observed when using the independent set reordering, as the scatter plot also shows that the reordering effect was not very pronounced, while we observe different results using Cuthill-McKee.

The Cuthill-McKee algorithm reduces the bandwidth of the matrix concentrating non-zero elements closer to the main diagonal and this reduction can significantly accelerate operations on large sparse datasets.

Reducing the bandwidth may not alter efficiently the structural relationships between users and movies, so the recommendations might not be optimal in every situation.

The Independent Set Ordering algorithm influence the independent structure between users or movies, aiming to reduce dependencies among elements. This structure improve recommendation quality especially in situation where there are clusters of users with similar rating behaviors. This method results to be more computationally expensive than Cuthill-McKee, but may produce more reliable results.

The Original Matrix can be a good choice if the matrix is already well-suited for the SVD algorithm and further optimization is not necessary.

However, if the matrix is highly sparse or poorly structured, the SVD algorithm might be less efficient producing not optimal recommendations.

If computation time is a priority and the dataset is very large, the Cuthill-McKee algorithm could be the better choice, as it tends to reduce computational complexity. If the quality of recommendations is more important and you can afford to invest additional computation time, Independent Set Ordering might produce better results, especially in scenarios with

complex user or movie relationships. If the matrix is already well-structured, using the original matrix might be the best compromise.

TABLE I
TOP 5 RECOMMENDED MOVIES FOR CUTHILL-McKEE REORDERED MATRIX

Movie Title	Movie ID	Predicted Rating
Amityville Curse, The (1990)	441	4.82
Island of Dr. Moreau, The (1996)	930	4.79
Loch Ness (1995)	838	4.78
Microcosmos: Le peuple de l'herbe	633	4.65
Secret of Roan Inish, The (1994)	462	4.62

TABLE II
TOP 5 RECOMMENDED MOVIES FOR ORIGINAL MATRIX

Movie Title	Movie ID	Predicted Rating
Breakdown (1997)	294	2.33
Crow: City of Angels, The (1996)	742	1.85
Brothers McMullen, The (1995)	25	1.82
Supercop (1992)	127	1.80
Emma (1996)	282	1.80

TABLE III
TOP 5 RECOMMENDED MOVIES FOR INDEPENDENT SET REORDERED MATRIX

Movie Title	Movie ID	Predicted Rating
Breakdown (1997)	294	2.33
Crow: City of Angels, The (1996)	742	1.85
Brothers McMullen, The (1995)	25	1.82
Supercop (1992)	127	1.80
Emma (1996)	282	1.80

VII. DISCUSSION

The analysis demonstrates that the choice of sparse matrix format significantly impacts performance. Among the tested formats, CSR and CSC formats resulted to be the most efficient with the MovieLens dataset. The CSC format, in particular, emerged as the most efficient choice for SVD operations, completing the task in just 0.193 seconds, thanks to its optimized access patterns and compact storage structure.

In contrast, the DOK format consistently performed the worst across all tests due to its internal structure. The DOK format stores non-zero elements in a dictionary, where the keys represent the coordinates (i, j) of the matrix entries, and the values correspond to the actual data. This structure makes it suitable for scenarios requiring frequent updates, while it is less effective for computationally intensive tasks. Consequently, while DOK is suitable for tasks requiring frequent updates, it is not optimized for numerical computations like matrix-vector multiplication or SVD, which involves iterating through the dictionary.

Formats like CSR and CSC, which store data in contiguous compressed arrays, are specifically designed to handle such operations more efficiently.

The LIL (List of Lists) represents each row as a list of column indexes and their corresponding values, making it straightforward to update. However, its performance diminishes for large-scale datasets or computationally intensive tasks, as it lacks the compactness and optimized access patterns of CSR and CSC.

Similarly, the COO (Coordinate) format is simple to implement and is particularly efficient for constructing sparse matrices. By storing non-zero elements as arrays of row indexes, column indexes, and values, COO allows for quick matrix assembly. However, its lack of optimized storage structures makes it less suitable for arithmetic operations compared to CSR and CSC.

These findings underscore the importance of selecting a storage format tailored to the specific needs of the application. The choice of format should consider the trade-offs between memory usage, update flexibility, and computational efficiency.

Reordering methods, like Reverse Cuthill-McKee and Independent Set Ordering, are important for improving the efficiency of sparse matrix computations, even if their benefits are not immediately evident in this specific project.

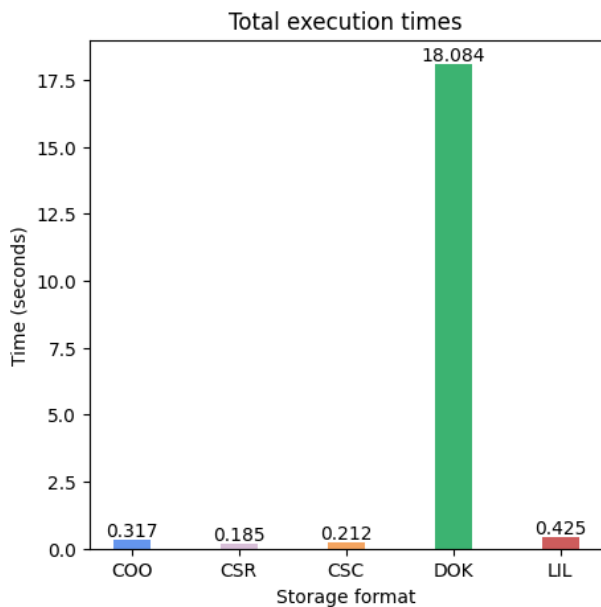


Fig. 10. Total Execution time for different sparse matrix formats.

REFERENCES

[1] Michael D. Ekstrand, John T. Riedl, and Joseph A. Konstan. Collaborative filtering recommender systems. *Foundations and Trends in Human-Computer Interaction*, 2010. Cap 2.4.

[2] Xiao Peng, Shao Liangshan, and Li Xiuran. Improved collaborative filtering algorithm in the research and application of personalized movie recommendations. In *2013 Fourth International Conference on Intelligent Systems Design and Engineering Applications*, pages 349–352, 2013.

[3] GroupLens Research. Movielens 100k dataset, 1998. Available at: <https://grouplens.org/datasets/movielens/100k/>.

[4] Rouhia M. Sallam, Mahmoud Hussein, and Hamdy M. Mousa. An enhanced collaborative filtering-based approach for recommender systems. *International Journal of Computer Applications*, July 2020.

TABLE IV
PERFORMANCE METRICS FOR DIFFERENT STORAGE FORMATS

Storage Format	Memory Usage (KB)	Creation Time (s)
COO	1562.48	0.05
CSR	1175.55	0.063
CSC	1178.43	0.026
DOK	1562.48	0.287
LIL	1171.86	0.061

Storage Format	Multiplication Time (s)	SVD Time (s)
COO	0.0017	0.265
CSR	0.0003	0.122
CSC	0.0003	0.183
DOK	0.1349	17.663
LIL	0.0029	0.361